

CUDA

Date 6 / 10 / 26

Clear Distinction:

int x = 10;

int *P = &x;

x → variable storing value of 10

&x → gives the memory address of x

Ex) 1000 10
 ↓ ↓
 mem add. value

P → Now, P has the memory address variable of x.

*P → Pointer, that says, go & fetch the value at memory address of x ⇒ 10

Cudamalloc:

Here, we create a empty memory space and a pointer variable. This empty memory space belongs to GPU memory.

Now, we make our Pointer Variable *P

A wise person does not entrust all his eggs to one basket.

Point to that empty memory space address.

```
int * &_x; // Creating Pointer Variable
cudaMalloc(&_x, sizeof(int));
```

⇓

We create an empty space to store one integer in the GPU memory, then, store that address in $\&_x$.

Cuda Memory:

It is the process of copying a variable data from the CPU to the GPU, then doing the process over there.

```
int x = 10; // Variable to be copied
int * &_x; // Pointer Variable
int result; // Printing value
```

```
cudaMalloc(&_x, sizeof(int));
```

// This is where the important processes start, first we create an empty

A man can succeed at almost anything for which he has unlimited enthusiasm.

// Memory space in the GPU. ~~the~~ point that
 // space using $q-x$.

// ($q-x \rightarrow$ empty memory space in GPU)

CudaMemcpy ($q-x$, $2x$, size of (int),
 CudaMemcpy Host to Device);

// This is (Destination of copy, source of
 copy)

// $q-x$ is the empty memory space where
 we want to store the value copy.

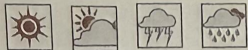
// $2x$ is the memory location from
 where the value must be fetched

// Now the value of $x = 10$, will be copied
 into the empty mem space, i.e.
 $q-x = 10$.

CudaMemcpy (result, $q-x$, size of (int),
 cudaMemcpy Host to Device);

// (Now this is the confusing part.

If you have ideas then you have the main asset to run your business and your life.



Mo Tu We Th Fr Sa Su

Date / /

// This is how we copy the data back from GPU to CPU. But, here we have used signs vice versa

(2 result, 4-x)

Reason for this is simple, we know that, initially, we created empty memory space & marked 2 & 4-x to it meaning, 4-x corresponds to that memory location. So, we don't wanna give it again, since computer knows what 4-x points to & where to fetch from. Therefore, in both places, we have given as just 4-x.

If cudaMalloc not mentioned then it would just be:

(2 result, 2 4-x, sizeof(int), cudaMem)

We need to give this since the computer needs to know what memory location the data has to be fetched from.

We lost yesterday somewhere between sunrise and sunset.